DAY 05: JAVA CONCURRENCY II

Data Bytes Technologies · Java 25 & Modern Stack

Yesterday you made blocking cheap with **virtual threads**, and ended on a cliffhanger: a counter that *lost count* when many threads touched it. Today we **solve** that, and learn the small set of tools that make shared state safe.

// WHERE WE ARE

- **Day 4**: virtual threads make blocking cheap: one task per request, *millions* of them.
- **Day 4's cliffhanger**: many threads sharing one mutable variable → a **race condition**.
- **Day 5**: today: *why* races happen, and how to make shared state safe (or avoid it entirely).

> 💡 Virtual threads gave us **scale**. They did **not** make concurrency safe. Cheap threads mean *more* code runs at once, so the sharing problem is bigger, not smaller.

// BY THE END OF TODAY, YOU CAN...

- explain **why a race happens**: *visibility* and *happens-before* in the Java Memory Model
- make shared state safe **three ways** (`synchronized`, **atomics**, **concurrent collections**) and pick on purpose
- **avoid the problem entirely** with immutability and confinement (the best strategy)
- spot and prevent **deadlock, livelock, and starvation**
- group subtasks with **structured concurrency** (preview) and carry context with **scoped values** (final)

// THE CLIFFHANGER, RECAPPED: A COUNTER THAT LOSES COUNT

```
// Day 4 left us here: 1,000 virtual threads, one shared mutable counter…
static int count = 0;

try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000; i++) {
        ex.submit(() -> count++);     // looks innocent
    }
}                                     // close() waits for all 1,000 tasks
System.out.println(count);            // expected 1000, but it's often LESS
```

No exception. No crash. Just a **wrong answer**, sometimes: the worst kind of bug, because it hides until production load.

// 🧪 TRY IT NOW (3 MIN): WATCH IT LOSE COUNT

In `jshell`, run the race a few times and watch the number fall below 1,000,000:

```
jshell> int[] count = {0};
jshell> try (var ex = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
   ...>     for (int i = 0; i < 1_000_000; i++) ex.submit(() -> count[0]++);
   ...> }
jshell> count[0]            // run the whole block 2–3 times, different, all < 1000000
```

Different answer each run, never the full million. That non-determinism **is** the race. You are watching updates get lost in real time.

// WHY DOES IT EVEN HAPPEN? `count++` IS THREE STEPS

The increment that looks like one action is really **read → modify → write**. Two threads can interleave between those steps.

- Thread A reads `count` → `0`
- Thread B reads `count` → `0` (before A has written)
- Both compute `0 + 1`, both write `1`

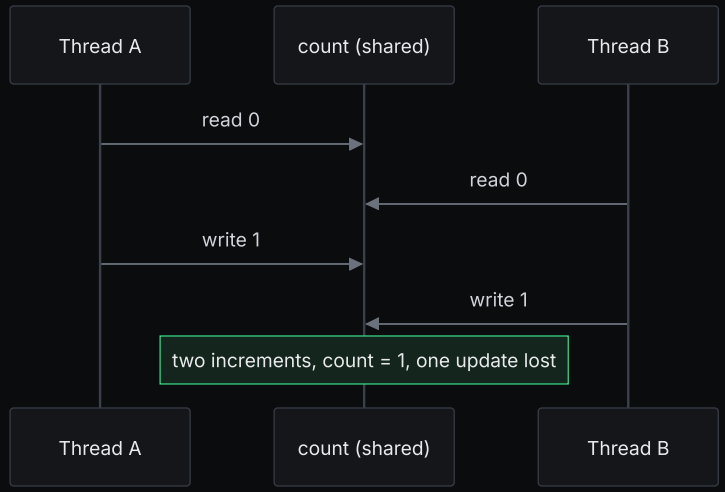
Two increments happened; the counter moved by **one**. That lost update is a **data race**: unsynchronised access to shared mutable state, where at least one access is a write.

// THE JAVA MEMORY MODEL, IN PLAIN TERMS

> 💡 **In plain terms**: each thread may keep its own working copy of a value (in a CPU cache or register). Without a synchronisation rule, a write by one thread is **not guaranteed to be visible** to another, and the order operations *appear* to happen in can differ between threads.

- **Visibility**: did your write actually reach the other thread, or is it reading a stale copy?
- **Ordering**: the compiler and CPU may reorder instructions that look independent.

// ONE PICTURE: THE LOST UPDATE



Takeaway: the bug is not in `count++`. It is in **two threads doing it at once with no rule** about who goes when.

// `HAPPENS-BEFORE`: THE RULE THAT FIXES VISIBILITY *AND* ORDERING

> 💡 **In plain terms**: if action X *happens-before* action Y, then X's effects are **visible** to Y, and X is **ordered** before Y. Every safe-publishing tool today exists to create this relationship.

- unlocking a lock *happens-before* the next thread locking it
- a write to a `volatile` field *happens-before* a later read of it
- everything a thread does before `Thread.start()` is visible to the new thread

You rarely reason about it by hand. You reach for a tool (a lock, an atomic, a concurrent collection) that **establishes** happens-before for you.

// 🧪 TRY IT NOW (2 MIN): DOES `VOLATILE` FIX THE COUNTER?

A tempting "fix": mark the field `volatile` and re-run the race. **Predict the result first:**

```
// imagine:  static volatile int count = 0;   then 1,000,000 × count++
// will it now reach exactly 1,000,000?
```

- `volatile` guarantees **visibility**: every thread sees the latest value.
- It does **not** make `count++` atomic. It is still read-modify-write, so updates still get lost.

Lesson: visibility ≠ atomicity. The counter needs the whole increment to be **indivisible**. That is what the next tools give us.

// MAKING IT SAFE #1: `SYNCHRONIZED` & INTRINSIC LOCKS

Every Java object has a built-in **intrinsic lock** (a *monitor*). `synchronized` acquires it, so only one thread runs the guarded code at a time, and it establishes happens-before on the way out.

```
// method form: locks on `this`
private int count = 0;
public synchronized void bump() { count++; }

// block form: lock on a private, dedicated object (preferred, narrow + private)
private final Object lock = new Object();
public void bump() {
    synchronized (lock) { count++; }   // critical section: one thread at a time
}
```

The code inside is a **critical section**. Keep it as small as possible. A lock you hold is a queue everyone else waits in.

// THE SAME COUNTER, THREE SAFE WAYS

The race line, and three ways to make it correct. Same goal, different trade-offs.

```
// Fix 3 - ConcurrentHashMap: atomic per-key accumulation
var counts = new ConcurrentHashMap<String, Long>();
counts.merge("orders", 1L, Long::sum);    // safe counting across many keys
```

All three are **correct**. They differ in cost and shape. That's the rest of this morning.

// 🧪 TRY IT NOW (6 MIN): FIX THE RACE THREE WAYS

Take the racing million-increment loop and make each version come out **exactly 1,000,000**:

```
// A) synchronized
Object lock = new Object();  long[] c = {0};
// ex.submit(() -> { synchronized (lock) { c[0]++; } });

// B) AtomicInteger
var ai = new java.util.concurrent.atomic.AtomicInteger();
// ex.submit(ai::incrementAndGet);

// C) ConcurrentHashMap
var m = new java.util.concurrent.ConcurrentHashMap<String,Long>();
// ex.submit(() -> m.merge("k", 1L, Long::sum));
```

Run each several times. The number stops wobbling: **correct, every time.** Feel the difference between "usually right" and "always right".

// ✋ EVERYONE WITH ME?

You should, right now, be able to say:

- a race is **shared + mutable + concurrent + a writer**. Remove any one and it's gone
- `volatile` gives **visibility**, not **atomicity**. It can't fix a counter alone
- `synchronized`, atomics, and concurrent collections each made the count **correct**

If the "visibility vs atomicity" line is fuzzy, flag it now. Every tool after this rests on it.

// THE CATCH: `SYNCHRONIZED` PINS VIRTUAL THREADS

Remember pinning from Day 4? A virtual thread that **blocks inside** `synchronized` cannot unmount. It holds its expensive carrier hostage.

So: `synchronized` is fine for **short, CPU-only** critical sections. If you might **block** while holding the lock (a DB call, a remote call), prefer `ReentrantLock`.

// `java.util.concurrent.locks`: EXPLICIT, FLEXIBLE LOCKS

`ReentrantLock` does what `synchronized` does, plus your virtual thread can unmount while it waits, and you can refuse to wait forever.

```
private final ReentrantLock lock = new ReentrantLock();

public void bump() {
    lock.lock();
    try { count++; }
    finally { lock.unlock(); }          // ALWAYS unlock in finally
}

// tryLock: take the lock only if it's free (or within a timeout), never hang
if (lock.tryLock(50, TimeUnit.MILLISECONDS)) {
    try { count++; }
    finally { lock.unlock(); }
} else {
    // couldn't acquire in time, back off, retry, or report "busy"
}
```

// ATOMICS & CAS, IN PLAIN TERMS

> 💡 **In plain terms**: an **atomic** wraps a value and offers indivisible updates with **no lock**. It leans on a hardware instruction, **compare-and-set (CAS)**: "set this to N+1, but *only* if it's still the N I just read, otherwise tell me, and I'll retry."

- No lock means no pinning, no deadlock, and great throughput for a single hot value.
- The cost: the retry loop spins under heavy contention: many threads, one value, lots of retries.

// ONE PICTURE: THE COMPARE-AND-SET RETRY LOOP

// ATOMICINTEGER VS LONGADDER

```
AtomicInteger count = new AtomicInteger();
count.incrementAndGet();            // lock-free CAS, perfect for one shared counter

// Under HEAVY write contention, LongAdder scales better:
LongAdder hits = new LongAdder();
hits.increment();                   // spreads writes across internal cells
long total = hits.sum();            // combine the cells only when you read
```

- `AtomicInteger` / `AtomicLong` / `AtomicReference`: one cell, CAS. Great default for a counter or flag.
- `LongAdder`: many cells, so threads rarely collide on writes; pay a small cost to `sum()` on read.

> 💡 Rule of thumb: **write-heavy, read-rarely** (metrics, counters) → `LongAdder`. Need the exact live value at every step → `AtomicInteger`.

// CONCURRENT COLLECTIONS: DON'T LOCK A `HASHMAP` BY HAND

The `java.util.concurrent` collections are built to be hit by many threads at once. Each operation is atomic and safely published, no external lock needed.

```
// ConcurrentHashMap: atomic per-key updates
var totals = new ConcurrentHashMap<String, Long>();
totals.merge(sku, 1L, Long::sum);         // read-modify-write for one key, atomically

// BlockingQueue: hand work between producer and consumer threads, safely
BlockingQueue<Order> queue = new LinkedBlockingQueue<>();
queue.put(order);             // producer - blocks if the queue is full
Order next = queue.take();    // consumer - blocks until something arrives
```

- `ConcurrentHashMap`: concurrent reads, atomic per-key writes; use `merge` / `compute`, not get-then-put.
- `BlockingQueue`: the backbone of producer/consumer pipelines; `put` / `take` handle the waiting.

// THE BEST STRATEGY: DON'T SHARE MUTABLE STATE AT ALL

Every fix so far *manages* sharing. The senior move is to **remove** the sharing. Then there is nothing to get wrong.

> 💡 **In plain terms**: a value that **never changes** (immutable) is safe to share with any number of threads. A value that is only ever touched by **one** thread (confined) needs no lock either.

- **Immutability**: `record`s and `final` fields: read by everyone, written by no one. No race possible.
- **Confinement**: keep mutable state inside one task/thread; share only immutable results.

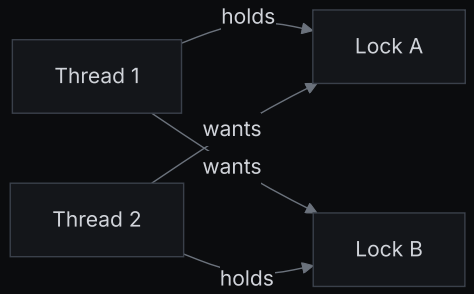
// IMMUTABILITY & CONFINEMENT, MADE CONCRETE

```
// Immutable value — safe to publish to any number of threads
record Quote(int stock, int priceCents) {}

// Confinement — each task owns its own work; only the result is shared
int total = files.stream().parallel()
        .mapToInt(this::countRowsInOneFile)   // each file counted in isolation
        .sum();                               // results merged by the framework

// Share results, not the mutable thing that produced them
List<Quote> snapshot = List.copyOf(liveQuotes);   // a frozen copy to hand out
```

Notice: no locks anywhere. The concurrency is real, but nothing mutable is shared, so there is nothing to synchronise.

// WHEN LOCKS BITE BACK: DEADLOCK

Two threads, two locks, acquired in **opposite orders**: each holds what the other needs, forever.



The program doesn’t crash. It **hangs**. No error, no progress. Just two threads waiting on each other until you kill it.

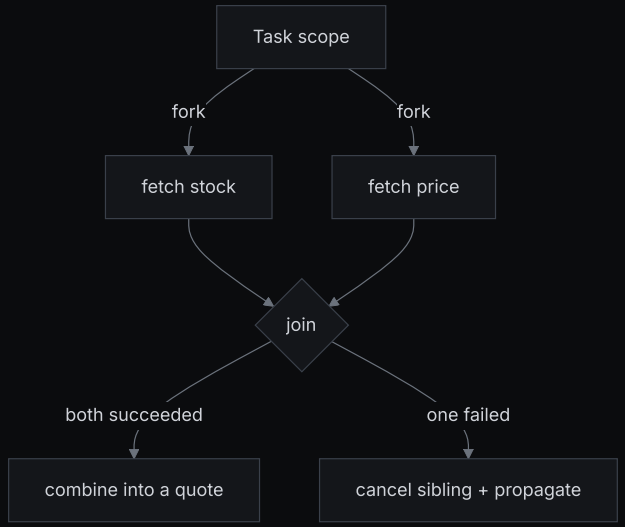
// DEADLOCK, LIVELOCK, STARVATION, AND HOW TO AVOID THEM

- **Deadlock**: a cycle of threads each waiting on a lock another holds. *Hangs.*
- **Livelock**: threads keep reacting to each other and retrying, but none makes progress. *Busy, stuck.*
- **Starvation**: one thread never gets the resource because others keep winning it. *Unfair.*

> 💡 The cures: acquire multiple locks in **one global order** (kills the cycle); use `tryLock` with a **timeout** (so you back off instead of hanging); hold locks **briefly**; and best of all, **hold fewer locks**: immutability and confinement again.

// DOING SEVERAL THINGS AT ONCE, AS ONE UNIT

Fan out two calls (fetch **stock** and **price**) then combine. With a raw executor, if one call fails the other keeps running: a **leaked** task doing wasted work.



Structured concurrency binds every subtask's lifetime to the scope that started it, the gift `try` / `finally` gave to resources, now for concurrency.

// STRUCTURED CONCURRENCY IS PREVIEW: HANDLE WITH CARE

> 💡 **In plain terms**: in **Java 25** this is a **preview** feature: compile and run with `--enable-preview --source 25`. The API was reworked. You now `open()` a scope (optionally with a `Joiner`) instead of subclassing the old `ShutdownOnFailure`.

```
// Java 25 PREVIEW — needs --enable-preview --source 25
try (var scope = StructuredTaskScope.open()) {          // default: all subtasks must succeed
    Subtask<Integer> stock = scope.fork(() -> fetchStock(sku));
    Subtask<Integer> price = scope.fork(() -> fetchPrice(sku));
    scope.join();                                        // waits; on failure cancels sibling & throws
    return new Quote(stock.get(), price.get());          // both succeeded, read results
}
```

For **production today**, the stable choice is a virtual-thread executor plus `Future`. Teach the concept; ship the stable shape until the API finalises.

// SCOPED VALUES: CONTEXT THAT FOLLOWS A VIRTUAL THREAD (FINAL)

Deep code often needs ambient context (a **correlation id**, a user, a tenant) without threading it through every method. The old tool, `ThreadLocal`, is mutable and a poor fit for a *million* threads.

```
private static final ScopedValue<String> CORRELATION_ID = ScopedValue.newInstance();

ScopedValue.where(CORRELATION_ID, "req-42").run(() -> handle(request));
// anywhere down the call stack, within that run():
String id = CORRELATION_ID.get();
```

- **Immutable + auto-unbinding**: bound for the duration of the call, gone the instant it returns.
- **Final in Java 25**, and it fits virtual threads: nothing to copy per thread, and **subtasks inherit the binding** automatically inside a `StructuredTaskScope`.

// 🧪 TRY IT NOW (2 MIN): CONTEXT WITH NO PARAMETER

In `jshell`, bind a value and read it from a helper that never received it as an argument:

```
jshell> ScopedValue<String> USER = ScopedValue.newInstance();
jshell> Runnable greet = () -> System.out.println("hi " + USER.get());
jshell> ScopedValue.where(USER, "ada").run(greet);
jshell> USER.isBound()
```

`greet` printed `"ada"` without being passed it, and `USER.isBound()` is `false` outside the `run`. The value lives exactly as long as the call, then vanishes.

// `CompletableFuture`, BRIEFLY: COMPOSING ASYNC RESULTS

Sometimes you want to **describe a pipeline** of dependent async steps without blocking a thread at each stage. `CompletableFuture` composes futures into a graph.

```
CompletableFuture<Integer> stock = CompletableFuture.supplyAsync(() -> fetchStock(sku));
CompletableFuture<Integer> price = CompletableFuture.supplyAsync(() -> fetchPrice(sku));
CompletableFuture<Quote> quote   = stock.thenCombine(price, Quote::new);   // when BOTH done
```

💡 Useful library glue, but it **colours** your code (everything becomes `CompletableFuture<…>`) and error handling gets fiddly. With virtual threads, plain blocking calls are usually clearer. Reach for `CompletableFuture` mainly to **compose** existing async APIs.

// COMMON TRAPS

- `volatile` **to fix a counter**: gives visibility, not atomicity. Use an atomic or a lock.
- `get` **then** `put` **on a** `ConcurrentHashMap` : two ops, still races. Use `merge` / `compute` .
- **Blocking inside** `synchronized` : pins the virtual thread. Prefer `ReentrantLock` (`unlock` in `finally`).
- **Locks in inconsistent order**: invites deadlock. Pick one global order; consider `tryLock` + timeout.
- **Reaching for a lock first**: ask whether the state can be **immutable or confined** instead.

// RECAP, IN PLAIN TERMS

Today you turned "concurrency is scary" into a small, reliable toolkit:

- a **race** is shared + mutable + concurrent + a writer, and the JMM (visibility, happens-before) explains it
- make it safe: `synchronized`, **atomics/CAS**, **concurrent collections**. Pick by contention and blocking
- better: **immutability + confinement**. Remove the sharing and there's nothing to guard
- avoid **deadlock** with lock ordering and timeouts; group work with **structured concurrency** (preview); carry context with **scoped values** (final)

The dividing line to remember: virtual threads and scoped values are **final**; structured concurrency is **preview**.

// TODAY'S LAB

🧪 **Make it safe, then make it structured**: take a racing counter and fix it (your choice of `synchronized`, `AtomicInteger`, or `ConcurrentHashMap`), prove it's correct, then use a `StructuredTaskScope` to fan out two slow calls and combine them as one unit.

Brief and starter: `labs/day-05/` · Structured concurrency needs `--enable-preview --source 25`.

Acceptance: the counter is **exactly** right every run, and the structured fetch returns a combined result (and cancels the sibling if one subtask fails).

// TAKEAWAY

Races aren't magic. They're **shared mutable state without a rule**. Give it a rule (a lock, an atomic, a concurrent collection), or better, **remove the sharing** with immutability and confinement. Reach for a lock last, not first.

Tomorrow, Day 6: PostgreSQL. A database has its *own* concurrency story: many clients hitting the same rows at once. The cure has different names (**transactions** and **isolation levels**) but it's the same problem you solved today, one layer down.